

Clean Code - Concurrency Notes (Complete Guide)

"Objects are abstractions of processing. Threads are abstractions of schedule." – James O. Coplien

1. Why Concurrency?

Concurrency is about decoupling WHAT gets done from WHEN it gets done. It improves performance, scalability, and system design when used correctly.

2. Race Condition Example

Bad Code (Not Thread Safe)

```
class Counter {
    private int count = 0;

    public int increment() {
        return ++count;
    }
}
```

Problem: Multiple threads can modify 'count' at the same time → inconsistent result.

Good Code (Thread Safe)

```
class Counter {
    private int count = 0;

    public synchronized int increment() {
        return ++count;
    }
}
```

3. Single Responsibility Principle in Concurrency

Bad Design

```
class UserService {
    private int counter = 0;

    public synchronized void createUser() {
        counter++;
    }
}
```

Good Design

```
class Counter {
    private int value = 0;

    public synchronized int increment() {
        return ++value;
    }
}
```

```

    }

    class UserService {
        private Counter counter = new Counter();

        public void createUser() {
            counter.increment();
        }
    }
}

```

4. Avoid Shared Data

Bad Code

```

List<String> sharedList = new ArrayList<>();

Thread t1 = new Thread(() -> sharedList.add("A"));
Thread t2 = new Thread(() -> sharedList.add("B"));

```

Good Code

```

List<String> list1 = new ArrayList<>();
List<String> list2 = new ArrayList<>();

```

5. Producer-Consumer Pattern

```

BlockingQueue<Integer> queue = new ArrayBlockingQueue<>(10);

new Thread(() -> {
    try { queue.put(1); } catch (Exception e) {}
}).start();

new Thread(() -> {
    try { int val = queue.take(); } catch (Exception e) {}
}).start();

```

6. Deadlock Example

Bad Code (Deadlock Risk)

```

synchronized(lock1) {
    synchronized(lock2) {}
}

synchronized(lock2) {
    synchronized(lock1) {}
}

```

Good Code (Consistent Lock Order)

```

synchronized(lock1) {
    synchronized(lock2) {}
}

```

7. Keep Critical Sections Small

Bad Code

```
synchronized(this) {  
    doHeavyWork();  
}
```

Good Code

```
doHeavyWork();  
synchronized(this) {  
    updateSharedState();  
}
```

8. Testing Concurrent Code

Example: Forcing Context Switch

```
Thread.yield();  
Thread.sleep(1);
```

9. Concurrency Issues Across Environments (Threads vs CPU Cores)

Concurrent code can behave differently depending on the environment. Differences in CPU cores, operating systems, and thread scheduling policies can expose hidden bugs.

Example Problem

Code that works fine on a laptop (4 cores) may fail on a server (16 cores) because more threads run truly in parallel.

Why This Happens

- More CPU cores = more true parallel execution - Different OS = different scheduling strategy - Timing differences expose race conditions

Bad Assumption

```
if (value == 0) {  
    value = 1;  
}
```

Looks safe, but without synchronization this can break under high concurrency.

Best Practices

- Always test on multiple environments - Run with more threads than CPU cores - Never assume execution order - Treat environment differences as risk factors

10. Important Quotes

"Concurrency bugs are not repeatable."

"Correct concurrency is complex, even for simple problems."

"Do not ignore one-off failures."